

1. Perceptron (Linearly Separable Data)

```
from sklearn.datasets import make_classification
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = make_classification(n_samples=200,
                          n_features=2,
                          n_redundant=0,
                          n_clusters_per_class=1,
                          random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = Perceptron(max_iter=1000, eta0=0.1, random_state=42)
model.fit(X_train, y_train)

prediction = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, prediction))
print("Weights:", model.coef_)
print("Bias:", model.intercept_)
```

```
Accuracy: 0.85
Weights: [[ 0.39007782 -0.10906549]]
Bias: [-0.1]
```

2. Multilayer Perceptron (Backpropagation)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42)

model = MLPClassifier(hidden_layer_sizes=(20,),
                      max_iter=500,
                      learning_rate_init=0.01,
                      random_state=42)

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, prediction))
```

```
Accuracy: 1.0
```

3. Genetic Algorithm

```
import random

population = [random.randint(0, 31) for _ in range(6)]

def fitness(x):
    return x * x
```

```

for generation in range(5):
    population = sorted(population, key=fitness, reverse=True)

    print("Generation", generation)
    print(population)

    parent1 = population[0]
    parent2 = population[1]

    child = (parent1 + parent2) // 2

    if random.random() < 0.2:
        child += random.randint(-2, 2)

    population[-1] = child

```

```

Generation 0
[22, 14, 13, 12, 12, 8]
Generation 1
[22, 18, 14, 13, 12, 12]
Generation 2
[22, 20, 18, 14, 13, 12]
Generation 3
[22, 21, 20, 18, 14, 13]
Generation 4
[22, 21, 21, 20, 18, 14]

```

4. Deep Neural Network

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

X_train = X_train / 255.0
X_test = X_test / 255.0

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.3),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=3)

loss, acc = model.evaluate(X_test, y_test)

print("Accuracy:", acc)

```

```

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step
Epoch 1/3
1875/1875 ————— 9s 4ms/step - accuracy: 0.9058 - loss: 0.3224
Epoch 2/3
1875/1875 ————— 6s 3ms/step - accuracy: 0.9498 - loss: 0.1667
Epoch 3/3
1875/1875 ————— 7s 4ms/step - accuracy: 0.9607 - loss: 0.1297
313/313 ————— 1s 2ms/step - accuracy: 0.9713 - loss: 0.0939
Accuracy: 0.9713000059127808

```

5. Decision Tree

```

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, random_state=42)

model = DecisionTreeClassifier(max_depth=3)

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, prediction))

```

Accuracy: 1.0

6. CNN Image Classification

```

import tensorflow as tf
from tensorflow.keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

X_train = X_train.reshape(-1,28,28,1)/255
X_test = X_test.reshape(-1,28,28,1)/255

model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32,(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(10,activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train,y_train,epochs=3)

print(model.evaluate(X_test,y_test))

```

```

Epoch 1/3
1875/1875 ————— 33s 16ms/step - accuracy: 0.9433 - loss: 0.2031
Epoch 2/3
1875/1875 ————— 25s 13ms/step - accuracy: 0.9781 - loss: 0.0747
Epoch 3/3
1875/1875 ————— 24s 13ms/step - accuracy: 0.9826 - loss: 0.0583
313/313 ————— 2s 6ms/step - accuracy: 0.9820 - loss: 0.0599
[0.05987181514501572, 0.9819999933242798]

```

7. Reinforcement Learning (Q-Learning)

```

import gymnasium as gym
import numpy as np

env = gym.make("FrozenLake-v1", is_slippery=False)

Q = np.zeros((env.observation_space.n, env.action_space.n))

alpha = 0.8
gamma = 0.95

```



```

from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data,
    data.target,
    random_state=42)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = SVC(kernel='rbf')

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, prediction))

```

Accuracy: 1.0

10. NLP using RNN (LSTM)

```

import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=5000)

X_train = pad_sequences(X_train, maxlen=200)
X_test = pad_sequences(X_test, maxlen=200)

model = tf.keras.Sequential([
    tf.keras.layers.Embedding(5000, 64),
    tf.keras.layers.LSTM(64),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=2)

print(model.evaluate(X_test, y_test))

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz>
17464789/17464789 ————— 0s 0us/step
Epoch 1/2
782/782 ————— 87s 108ms/step - accuracy: 0.8081 - loss: 0.4219
Epoch 2/2
782/782 ————— 144s 111ms/step - accuracy: 0.8833 - loss: 0.2875
782/782 ————— 24s 30ms/step - accuracy: 0.8364 - loss: 0.4165
[0.4165094494819641, 0.8364400267601013]

